SPECTROview: An open-source application for spectroscopic data processing, fitting, and visualization

Van Hoan Le [a,*]

[a] Univ. Grenoble Alpes, CEA, LETI, Grenoble 38000, France
[*] Corresponding author. E-mail address: van-hoan.le@cea.fr

Abstract

SPECTROview is a free, open-source desktop application for interactive processing, curve fitting, and visualization of spectroscopic data. Written in Python with a PySide6 graphical interface, it addresses the needs of researchers and engineers who work with discrete spectra and hyperspectral datasets, including two-dimensional maps and semiconductor wafer maps, produced by spectroscopic techniques (such as Raman spectroscopy, photoluminescence, etc.). The application is organized into three core workspaces (Spectra, Maps, and Graphs) that cover the complete analytical workflow: 1D or 2D data import from multiple instrument formats, preprocessing with interactive baseline correction, multi-peak curve fitting, and publication-quality statistical plotting. A central contribution is the Vectorized Batch Fit Engine (VBF), a batched Levenberg-Marquardt optimizer that fits all spectra of a dataset simultaneously using vectorized NumPy operations and analytical Jacobians, achieving up to 60× speed up over conventional *lmfit* based per-spectrum approaches. SPECTROview is cross-platform, installable via pip, and released under the GPL-3.0 license.

Keywords

Spectroscopy; VBF engine; Hyperspectral data; Peak fitting; Data visualization; Open-source software

Metadata

Nr	Code metadata description	*Metadata*
C1	Current code version	*26.26.2*
C2	Permanent link to code/repository used for this code version	*https://github.com/CEA-MetroCarac/SPECTROview*
C3	Legal code license	*GNU General Public License (GPL-3.0)*
C4	Code versioning system used	*git*
C5	Software code languages, tools and services used	*Python*
C6	Compilation requirements, operating environments and dependencies	*Python between (3.8 & 3.12); PySide6, NumPy (< 2.0), SciPy, Pandas, Matplotlib, Seaborn (cf. pyproject.toml for the complete list).*
C7	If available, link to developer documentation/manual	*https://CEA-MetroCarac.github.io/SPECTROview/*
C8	Support email for questions	van-hoan.le@cea.fr

1. Motivation and significance

Spectroscopic techniques such as Raman scattering and photoluminescence are widely used across various application fields including materials science, chemistry, biology, geology, and semiconductor manufacturing. Modern spectrometers can acquire either individual spectra or hyperspectral datasets, in which a complete spectrum is recorded at each point of a two-dimensional grid, yielding maps containing thousands to tens of thousands of spectra. Extracting meaningful insights from raw spectroscopic datasets usually requires a multi-step workflow encompassing data import, spectral pre-processing (e.g., spectral range selection, cropping, and baseline subtraction), multiple-peak fitting, aggregation of best-fit results and statistical visualization.

In practice, this workflow is often fragmented across multiple software solutions. Data acquisition is typically performed using vendor-specific proprietary software, such as Renishaw WiRE or Horiba LabSpec, which generally offers limited flexibility for peak fitting, requires tedious manual result extraction and aggregation, provides minimal support for batch processing and custom statistical visualization, and is subject to restrictive licensing.

General-purpose open-source tools such as fityk [1] provide flexible curve fitting with parameter constraints but lack native support for hyperspectral data visualization and processing. Library RamanSPy [2] offer powerful preprocessing and machine-learning pipelines but lack a graphical user interface (GUI) for interactive peak fitting. Similarly, PyFasma [3] offers a Jupyter notebook-based API for Raman preprocessing with spectral deconvolution using lmfit package [4]. However, it also lacks a GUI, native hyperspectral map visualization, or batch-fitting capabilities. More recent python-based tools such as fitspy [5] (which also relies on lmfit library [4] for fitting features) support analysis of both individual spectra and hyperspectral datasets. However, their performance can degrade significantly for medium- and large-scale datasets because the fitting workflow remains largely based on sequential Python-level iteration. While multithreading and multiprocessing can provide partial acceleration, the Global Interpreter Lock (GIL) limits true

parallel execution of CPU-bound tasks, and multiprocessing incurs additional overhead from data serialization and inter-process communication. As a result, both scalability and computational efficiency remain challenging on standard multi-core workstations.

To the best of our knowledge, no existing free and open-source software package combines: (i) support for multiple spectroscopic data formats, (ii) interactive spectral preprocessing (e.g., cropping and baseline subtraction) and multi-peak fitting with parameter constraints, (iii) native handling of both discrete spectra and hyperspectral datasets, including 2D maps and semiconductor wafer maps, (iv) high-performance batch fitting optimized for large-scale datasets, and (v) integrated statistical analysis and publication-quality visualization within a single application.

SPECTROview was developed at the Platform for Nano-characterisation (PFNC) of CEA-Leti to address this gap. Designed by researcher-engineers for researcher-engineers, it streamlines the complete spectroscopic analysis workflow from raw data import to visualization within a unified and user-friendly environment. Its feature set is driven by the practical challenges of processing the large volumes of spectroscopic data generated daily in both academic research laboratories and semiconductor manufacturing environments. A key innovation of SPECTROview application is the Vectorized Batch Fit (VBF) engine, a high-performance implementation of the Levenberg-Marquardt algorithm specifically designed for large-scale hyperspectral datasets. Although batched optimization is widely employed in modern deep-learning frameworks, most open-source spectroscopy software still performs fitting sequentially, optimizing spectra one at a time. In contrast, the VBF engine reformulates the fitting of N spectra as a single tensor-based linear algebra problem implemented entirely in NumPy. Importantly, this vectorized approach preserves the independent optimization of each spectrum by maintaining spectrum-specific convergence states. A dynamic convergence mask progressively excludes converged spectra from subsequent iterations, thereby minimizing unnecessary computations throughout the fitting process. By combining the computational efficiency of batched numerical operations with the robustness of independent Levenberg-Marquardt optimization, the VBF engine achieves speedups of up to 60× compared with conventional sequential spectrum-by-spectrum fitting approaches. This performance improvement enables the routine analysis of large hyperspectral datasets on standard workstations, substantially reducing processing times while maintaining fitting accuracy.

2. Software description

2.1. Software architecture

SPECTROview is a Python-based application utilizing PySide6 (Qt 6) [6] for its graphical user interface (GUI). The application follows a strict Model-View-ViewModel (MVVM) architecture that separates data management (Model), business logic and application orchestration (ViewModel), and user interface layout (View). This modular design enables independent testing of the numerical core and simplifies future extensions of the user interface. Figure 1 illustrates the architecture and module interactions. A comprehensive architectural description is available in the developer guide: https://cea-metrocarac.github.io/SPECTROview/developer/

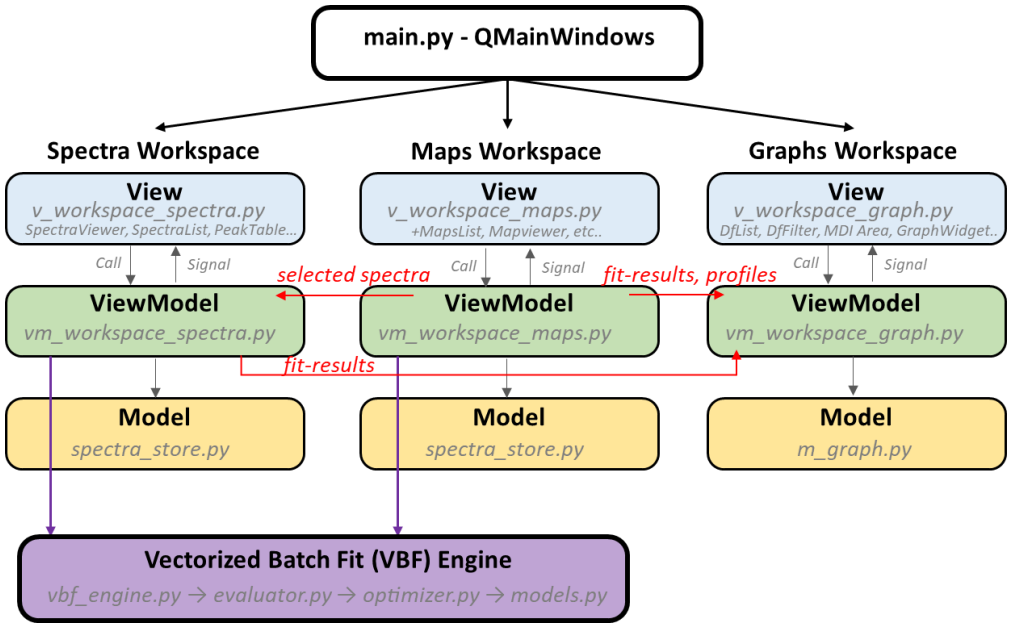


Figure 1: Simplified architecture diagram of SPECTROview with three workspaces (Spectra, Maps, Graphs), their MVVM layers, and inter-workspace communication.

Figure 2 present the application's user-interface organized into three workspaces, each implemented in a QTableWidget:

- Spectra Workspace: manages discrete spectra including loading, preprocessing, interactive peak fitting, and fit-result collection.
- Maps Workspace: inherits and extends the Spectra Workspace to handle hyperspectral datasets. It adds spatial heatmap visualization (2D maps and wafer maps) and includes dedicated list boxes to navigate between multiples loaded datasets and individual spectra within them.
- Graphs Workspace: provides a dataframe-based statistical plotting environment with support for eight plot styles and dynamic filtering. This allows users to work on a single datasheet across multiple plots.

To ensure a seamless workflow, cross-workspace communication is managed via Qt signal-slot connections and startup dependency injection. Key workflow examples include:

- Transferring best-fit results from the Spectra or Maps Workspace directly to the Graphs Workspace for immediate statistical analysis and visualization.
- Sending individual spectra from a 2D map to the Spectra Workspace for detailed inspection and inter-map comparison, bypassing the need for file-based exports.
- Exporting line profiles (extracted via two-point selection on a 2D heatmap) directly to the Graphs Workspace, transferring both the raw data and the resulting plot.

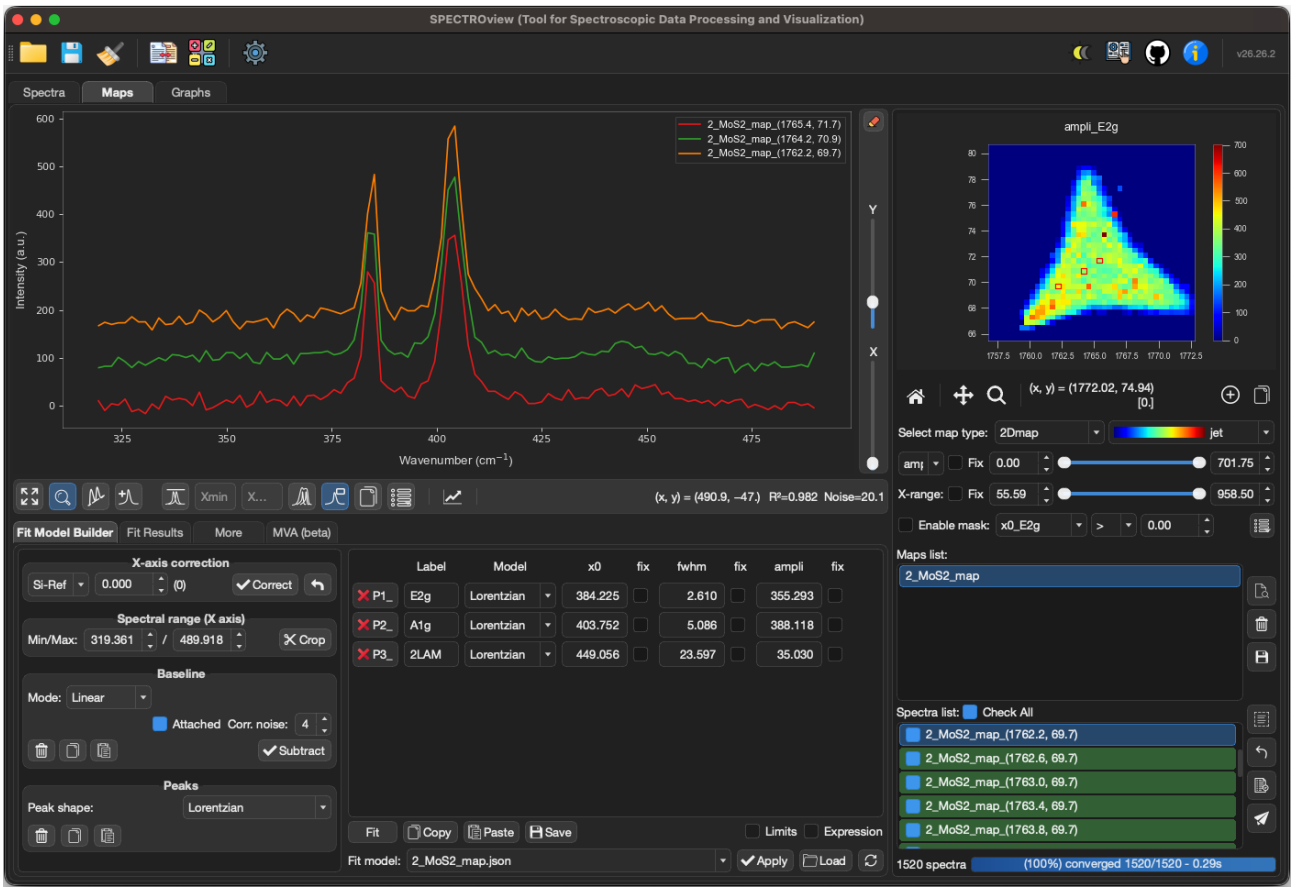


Figure 2: SPECTROview's main user interface, organized into three workspaces: Spectra, Maps, and Graphs. (An animated version of this figure can be found on the project's documentation page).

2.2. Software functionalities

2.2.1 Data import

Table 1 lists all file formats supported by SPECTROview. Examples of all supported data formats are available in the GitHub repository folder : examples/spectroscopic_data

Table 1: Supported file formats

Format	Extension	Loader	Data type
Delimited text	.txt, .csv	Auto-detected delimiter	Single spectra and maps
Renishaw WiRE	.wdf	renishawWiRE library	Single spectra and maps
SPC	.spc	Built-in binary reader	Single spectra and maps
TRPL	.dat	Built-in parser	Time-resolved decays
Tabular data	.xlsx, .csv	Pandas & Openpyxl	DataFrames for plotting

2.2.2 Preprocessing

The preprocessing pipeline operates on working copies of the raw data, preserving the original arrays for non-destructive analysis. Available operations include:

- X-axis correction (optional): linear shift of the x-axis.
- Spectral range selection: interactive cropping of the x-axis range.
- Baseline correction: both manual (linear, polynomial with user-placed anchor points) and automatic (arPLS, airPLS, asLS, etc.) baseline subtraction. Automatic baseline algorithms are provided by the pybaselines package [7] which leverages optimized NumPy/SciPy solvers [8], and can be previewed in real time before subtraction.

These operations are fully vectorized, allowing users to apply preprocessing steps to an individual spectrum or universally across all loaded maps.

2.2.3 Multiple peaks fitting with the VBF engine

Users interactively define peak models by clicking directly on the spectrum. Each peak is assigned a line shape and populated into a peak table, where initial parameters are auto-estimated and users can define bounds or inter-parameter constraints.

Fitting relies on the VBF engine, a custom batched Levenberg-Marquardt optimizer [9]. Rather than iterating over spectra in a Python loop, the VBF engine reformulates the simultaneous fitting of N spectra into coupled tensor operations:

- The N observed spectra are stacked into an (N × M) data matrix, where M is the number of spectral points.
- Initial parameter guesses are assembled into an (N × K) matrix, where K is the number of free parameters, with per-spectrum amplitude scaling.
- Model evaluation and Jacobian computation produce (N × M) and (N × M × K) tensors, respectively.
- Matrix multiplications and transpositions for the normal equations (J^TJ and J^Tr) are assembled and performed via *np.einsum,* and solved in a single LAPACK-dispatched *np.linalg.solve call* [10], yielding the parameter updates for all spectra simultaneously.

All built-in peak models utilize analytical Jacobians, avoiding the 2K extra function evaluations per iteration required by finite-difference methods. Despite batching, spectra still converge independently: the optimizer maintains a per-spectrum damping factor and a boolean convergence mask that progressively excludes converged spectra from subsequent iterations.

Additional performance optimizations include: (i) a template-based model application that pre-computes Model objects once and clones only lightweight parameter dictionaries, (ii) batch preprocessing that computes range masks and baseline indices once for spectra sharing the same x-axis, (iii) an adaptive solver that selects between batched *np.linalg.solve* [10] and Cholesky decomposition (*cho_solve*) [10] based on matrix dimensions, and (iv) noise-aware weight masking that ignores sub-noise-floor data to prevent ghost peaks.

Entire fit models including baseline settings, peak definitions, and parameter constraints can be copied, and pasted across loaded datasets, or can be saved as reusable analysis templates.

2.2.4 Hyperspectral map visualization

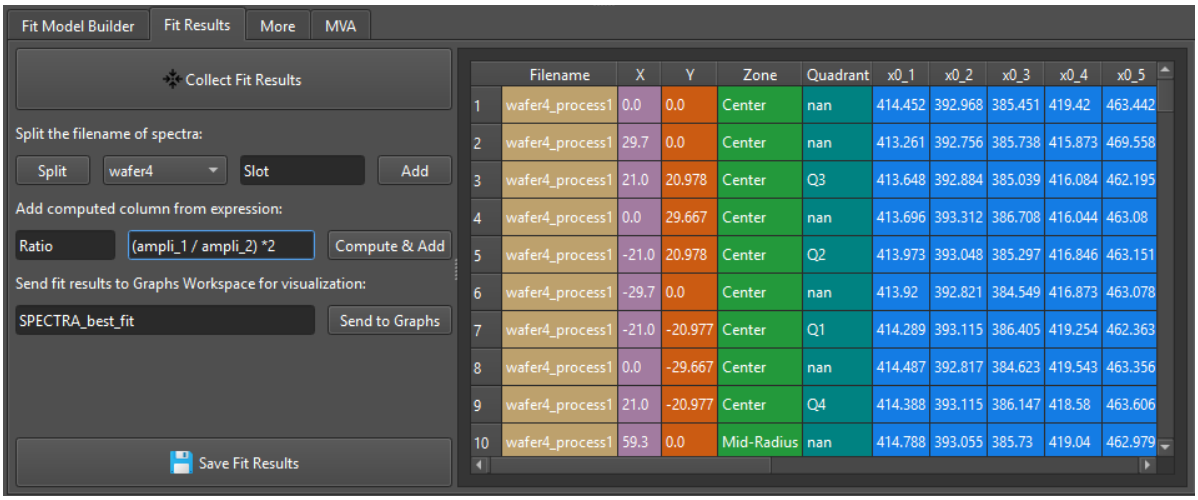
The Maps Workspace renders loaded hyperspectral datasets as interactive heatmaps (Figure 2). The Z-axis (color scale) can display raw integrated intensity, integrated area, or any best fit results (cf. section 2.2.5).

Users can interact with the heatmap to select individual or multiple spectra (via simple mouse click or hold Ctrl + clicking on the heatmap), define regional subsets using a rectangular drag selection, or extract spatial line profiles between two points. The selected spectra are immediately displayed in the plot widget for inspection and fitting, while extracted line profiles can be exported directly to the Graphs Workspace as DataFrames.

SPECTROview also supports wafer-map rendering using *scipy.griddata* interpolating the data onto a regular grid constrained by a circular mask.

2.2.5 Best-fit results aggregation and export

Once curve fitting has been applied across a set of discret spectra (Spectra Workspace) or entire hyperspectral maps (Maps Workspace), SPECTROview provides a streamlined interface to aggregate, transform, and export the results. Clicking the “Collect” button in the “Fit Results” tab instantly compiles all best-fit parameters (peak positions, amplitudes, fwhm, …) into a sortable Pandas DataFrame for direct GUI visualization (Figure 3).



	Filename	X	Y	Zone	Quadrant	x0_1	x0_2	x0_3	x0_4	x0_5
1	wafer4_process1	0.0	0.0	Center	nan	414.452	392.968	385.451	419.42	463.442
2	wafer4_process1	29.7	0.0	Center	nan	413.261	392.756	385.738	415.873	469.558
3	wafer4_process1	21.0	20.978	Center	Q3	413.648	392.884	385.039	416.084	462.195
4	wafer4_process1	0.0	29.667	Center	nan	413.696	393.312	386.708	416.044	463.08
5	wafer4_process1	-21.0	20.978	Center	Q2	413.973	393.048	385.297	416.846	463.151
6	wafer4_process1	-29.7	0.0	Center	nan	413.92	392.821	384.549	416.873	463.078
7	wafer4_process1	-21.0	-20.977	Center	Q1	414.289	393.115	386.405	419.254	462.363
8	wafer4_process1	0.0	-29.667	Center	nan	414.487	392.817	384.623	419.543	463.356
9	wafer4_process1	21.0	-20.977	Center	Q4	414.388	393.115	386.147	418.58	463.606
10	wafer4_process1	59.3	0.0	Mid-Radius	nan	414.788	393.055	385.73	419.04	462.979

Figure 3: The FitResult tab interface.

A built-in filename-splitting tool can automatically extract metadata from the file names (e.g., sample identifiers, process conditions, or measurement temperatures) and map them into new DataFrame columns, eliminating manual annotation. This feature is highly advantageous for users employing systematic file naming conventions during data acquisition.

Furthermore, users can generate derived columns by applying standard mathematical operators (+, -, *, /, **) to existing fit parameters (e.g., calculating peak shifts or intensity ratios), as shown in Figure 3.

The resulting DataFrame can be (i) sent directly to the Graphs Workspace for immediate statistical visualization (e.g., box plots, trendlines), or (ii) exported as an Excel (`.xlsx`) or CSV (`.csv`) file. This single-click collection and in-application transformation workflow removes the intermediate file-export steps that are typically required by other spectroscopy tools, thus significantly accelerating the path from raw data to publication-ready figures.

2.2.5 Statistical plotting

The Graphs Workspace operates on Pandas DataFrames loaded either from supported files (CSV, excel) or transferred from other workspaces. It supports various plot types: *point*, *scatter*, *box*, *bar*, *line*, *trendline*, *histogram*, *wafer*, and *2Dmap* plots. Visualizations are rendered using *Seaborn* [11] and *Matplotlib* [12] within a Multiple Document Interface (MDI, Qt) windows, enabling the simultaneous display and comparison of multiple plots (Figure 6). A graph customization dialog provides axis limits, annotations, and export to clipboard or image file.

For wafer datasets containing a *Slot* column, a batch wafer-plotting mode automatically generates multiple selected wafer plots at once. In addition, the intelligent “collect fit results” feature can automatically identify data originating from different wafer regions, such as quadrants or radial zones (center, mid-radius, and edge), based on the wafer diameter and XY coordinates of spectra (an example is showed in results data table in Figure 3). This functionality simplifies statistical analysis and comparison across multiple wafers and wafer’s regions using a single dataset, without requiring additional data cleaning or post-processing.

2.2.6 Multivariate analysis

SPECTROview includes built-in multivariate analysis capabilities: Principal Component Analysis (PCA) via singular value decomposition and Non-negative Matrix Factorization (NMF) via Lee-Seung’s multiplicative update rules [13]. Both methods operate on preprocessed spectra and produce scree plots, loadings, and score visualizations. Results can be exported to the Graphs Workspace for further analysis.

2.2.7 Save and resumes works

SPECTROview uses dedicated native file formats to persist the complete analysis state of each workspace. Clicking the "Save" button in the menu bar generates a workspace-specific archive:

- *.spectra* : stores all loaded 1D spectra, baseline configurations, fit models, and aggregated fit results.
- *.maps* : stores the raw 2D/wafer map data, instrument metadata, associated spectra, and point-by-point fit results.
- *.graphs* : stores all loaded DataFrames, plot configurations, and annotations.

To guarantee the rapid serialization of large datasets, the Spectra and Maps workspaces archives are ZIP-compressed, bundling JSON metadata with compressed NumPy arrays. The Graphs workspace archives utilizes a gzip-compressed JSON format.

Users can resume a session by opening or dragging and dropping a saved workspace file *(.spectra, .maps, .graphs*), directly into the main window. SPECTROview auto-detects the format,

navigates to the appropriate workspace, and restores the exact analysis state. This ensures strict reproducibility and seamless collaboration.

2.2.8 Other Utilities

In addition to its three core workspaces, SPECTROview integrates some auxiliary modules accessible from the main menu bar:

- Quick Calculators (Appendix A): A set of interactive scientific calculators designed designed for routine spectroscopy use in the laboratory. These include calculations for laser spot size and depth of focus, as well as optical penetration depth estimates based on laser wavelength and extinction coefficient, with direct access to the refractiveindex.info database and accompanying pedagogical illustrations. This module also includes a Unit Converter, enabling bidirectional conversions between wavelength (nm), energy (eV), and wavenumber (cm^{-1}), as well as Raman shift calculations for a given laser excitation wavelength.
- Hyperspectral Data Converter: A tool for converting hyperspectral text files into formats supported by SPECTROview.

These utilities are implemented as lightweight, self-contained dialog modules that can be invoked at any point during an analysis session without interrupting the current workspace state.

3. Illustrative examples

This section demonstrates a representative workflow using example datasets distributed with SPECTROview in the *examples/* folder of the GitHub repository. An intuitive User Manual with animated demonstrations of various functionalities, is integrated in the application (accessible via User Manual button in the main GUI menu bar). An online version is also available at https://cea-metrocarac.github.io/SPECTROview/user_manual/.

3.1. Fitting discrete spectra in Spectra workspace

Once the spectra are loaded, the user can process them through a sequence of steps: (i) defining a spectral region of interest, (ii) performing baseline correction using either manual or automatic modes, (iii) defining one or more peaks, and (iv) executing the fit.

Peaks are placed interactively by clicking directly on the spectrum plot. Each peak can be further configured by adjusting its line shape, parameter bounds, or inter-parameter expressions. Once all peaks are defined, clicking "Fit" dispatches the data to the VBF Engine, which fits the selected spectra simultaneously. The fitted curves, individual peak contributions, and residuals are then displayed in the spectrum viewer. Users can also load a predefined fit model from a previous session and click the "Apply" button to execute the entire fitting pipeline in a single action.

Figure 4 presents an example of a fitted MoS_2 spectrum with multiple peaks, exported directly from the application. The user can adjust several parameters before exporting, such as the figure's aspect ratio, peak labels, grid lines, legend entries, and individual spectrum colors. A comprehensive list of available view options is detailed in the User Manual.

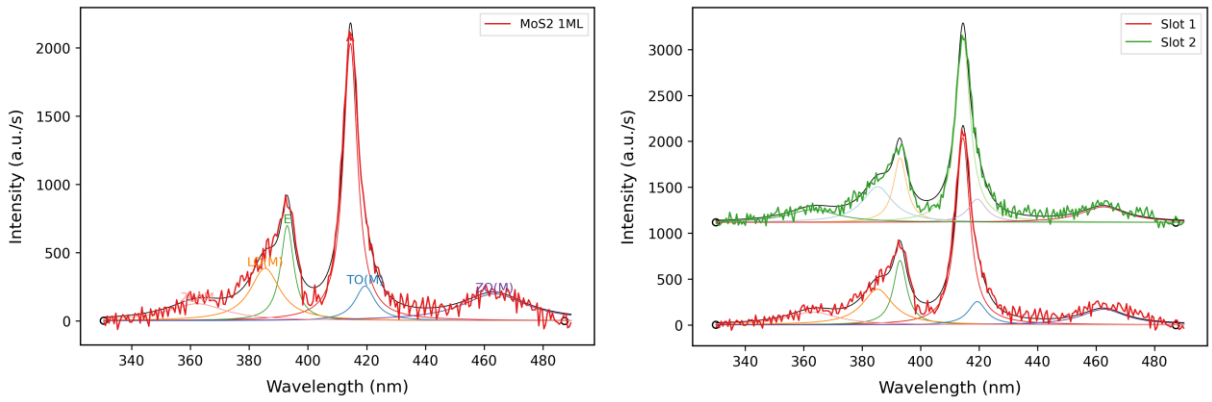


Figure 4: (a) Example of an MoS_2 spectrum after baseline subtraction and multi-peak fitting. (b) Multiple spectra selection and plotting with vertical shift for precise inspection and comparison.

3.2. Analyzing a hyperspectral map in Maps Workspace

As described in Section 2.1, the Maps Workspace inherits all core features of the Spectra Workspace, with adaptations to the user interface and processing logic designed to facilitate navigation between maps and between individual spectra within a given map. Once loaded, hyperspectral map files are listed in the MapList widget, and all individual spectra from the selected map populate the SpectraList widget (right panel, Figure 2). By default, the heatmap viewer displays the integrated intensity across the selected spectral range. The fitting workflow remains identical to that used in the Spectra Workspace (Section 3.1).

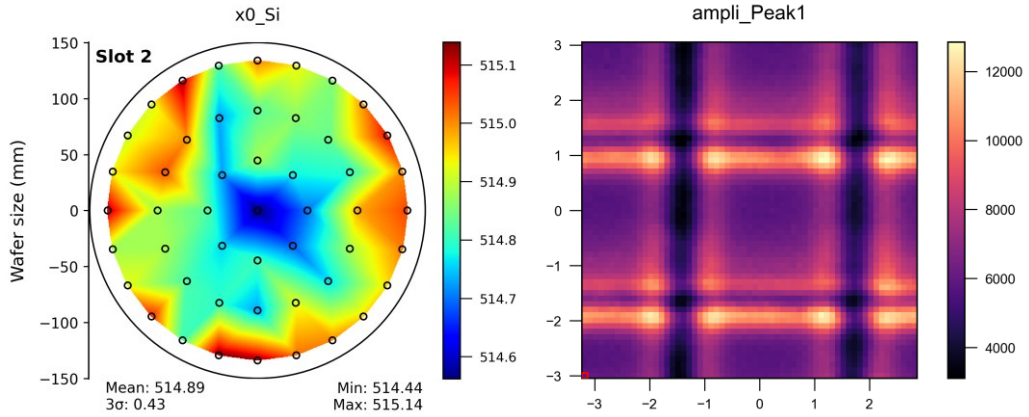


Figure 5: (a) Wafer map displaying peak position alongside automatically computed statistical summaries (mean, min, max, and standard deviation). (b) 2D intensity map of the hyperspectral dataset.

The VBF Engine processes the entire map (or all loaded maps if hold “Ctrl” key and clicking) as a single batch operation. Following the fit, the Z-parameter dropdown updates to include the newly fitted parameters (e.g., peak position, FWHM, amplitude). The heatmap is then re-rendered to visualize the spatial distribution of each fitted parameter across all defined peaks (Figure 5).

3.3. Plotting graphs

Figure 6 present the GUI of the Graphs workspace. Data for visualization can be loaded from external CSV for Excel files or imported directly from the best-fit results forwarded from the Spectra or Maps workspaces (Figure 3). Multiple datasets can be loaded and displayed in the

listbox located in the right-side panel (Figure 6). The user selects a dataset from the list panel, assigns columns to the X, Y, and optionally Z axes via dropdown menus, and chooses a plot style (e.g., scatter, box, bar, line, trendline, histogram, wafer, or 2D map). Clicking "Add Plot" renders the figure in the MDI area. Multiple plots can be displayed simultaneously for side-by-side comparison.

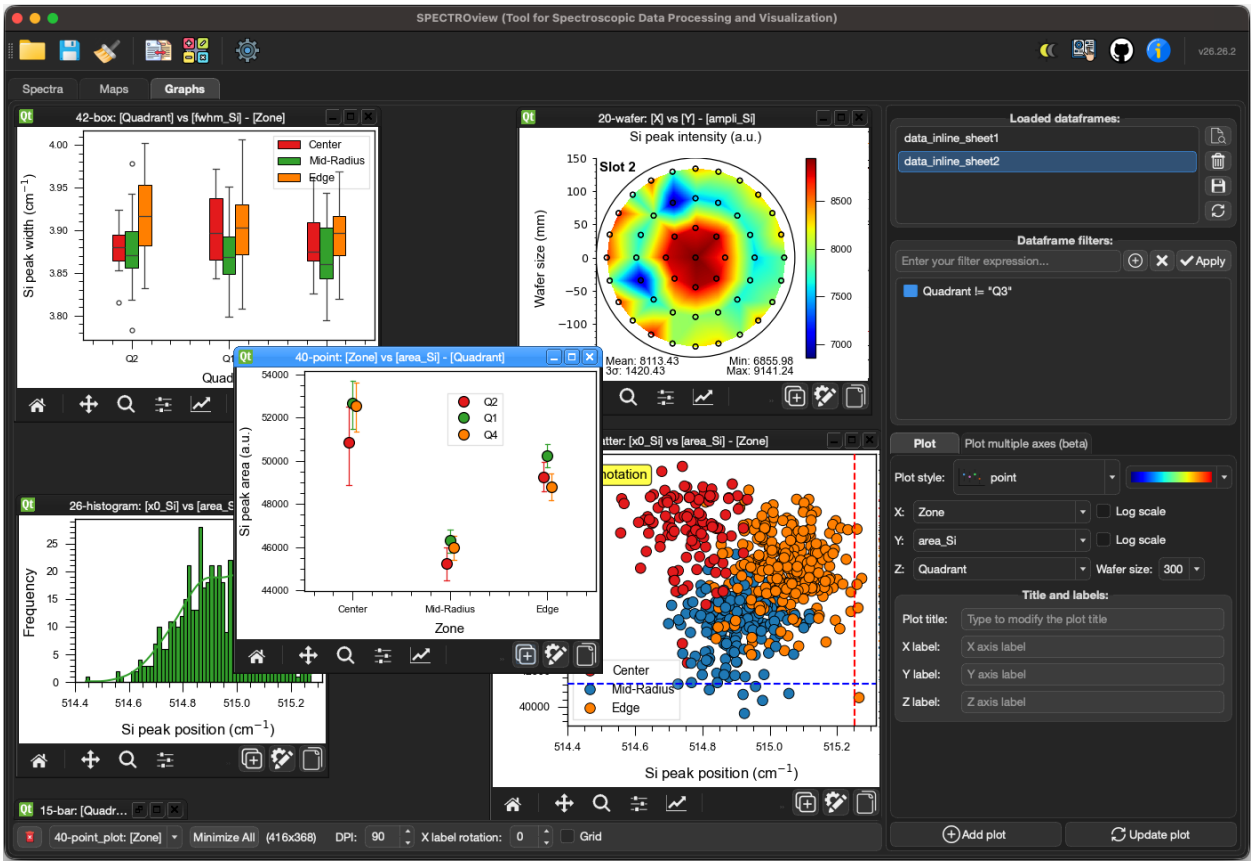


Figure 6: User interface of Graphs Workspace

Each plot can be further refined through a customization dialog that provides control over axis limits, legend entries (labels, colors, markers), annotations (draggable vertical/horizontal reference lines, text annotations), and other visual properties. As previously shown, Figure 5 provides an example of a wafer plot displaying fitted parameters alongside automatically computed statistics (mean, max, min, and standard deviation). Other examples are shown in Figure 6.

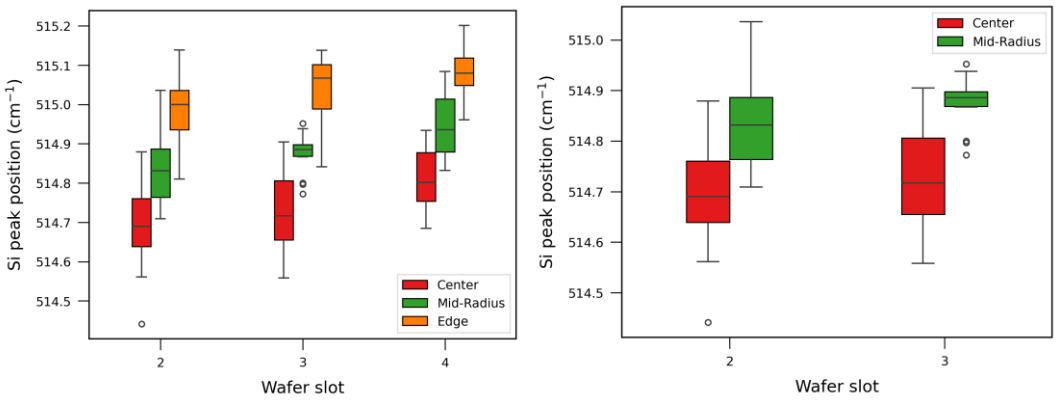


Figure 7: (a) Box plot generated directly from the best-fit results forwarded from the Maps workspace. It illustrates an inter-wafer comparison of the Si peak position, with the data divided into three distinct wafer regions prior to any post-processing or data cleaning. (b) The same data as in (a), but with dynamic filters applied to exclude Wafer 4 and the "Edge" zone from the plot.

Figure 7a presents a example of box plots for comparison between data from three wafers. A dynamic filtering feature (cf. right-side panel in Figure 6) allows the user to subset the plotted data using "filtering expressions" applied directly to DataFrame columns. This enables rapid exploration of parameter dependencies without modifying the underlying dataset. For example, the plot in, Figure 7b is obtained using same datasheet as Figure 7b, but applies two filters (*Slot != "4"* and Zone != "Edge") to exclude wafer n° 4 and wafer edge zone from the visualization. Additional demonstrations can be found in the online documentation: https://cea-metrocarac.github.io/SPECTROview/user_manual/graphs/.

4. Impact

4.1. Comparison with some existing tools

Table 2 present a comparison of SPECTROview with some existing spectroscopic fitting tools. SPECTROview distinguishes itself along two axes:

Table 2: Compares SPECTROview with representative open-source and proprietary tools for spectroscopic data analysis. NA = not applicable (library without fitting or proprietary implementation); ✓ = partial support ((fityk uses analytical derivatives for some built-in functions).*

Feature	SPECTROview	Fitspy	Fityk	RamanSPy	PyFasma	Wire/Labspec
Open source	✓	✓	✓	✓	✓	✗
Cross-platform	✓	✓	✓	✓	✓	✗
GUI for interactive fitting	✓	✓	✓	✗	✗	✓
Hyperspectral map support	✓	✓	✗	✓	✗	✓
Wafer map support	✓	✗	✗	✗	✗	✗
Batched vectorized fitting	✓	✗	✗	✗	✗	✓
Analytical Jacobians	✓	✗	✓*	NA	✗	NA

Integrated satistical plotting	✓	✗	✗	✗	✗	✗
Inter-peaks prameter expression	✓	✓	✓	✗	✓	✗
MVA analysis	✓	✗	✗	✓	✓	✓

First, it is the only open-source tool that integrates the complete workflow from multi-format file support (Table 1) through interactive peak fitting to publication-quality statistical plotting—within a single GUI application. Its ergonomic and highly optimized interface is designed to be accessible to a broad audience, including researchers, engineers, and students. It provides an efficient platform for spectral data inspection, as well as the interactive processing and analysis of both small and large-scale hyperspectral datasets, all without requiring the user to write code and, notably, free of any licensing barriers.

Second, its VBF Engine introduces a high-performance computational approach that is, to the best of our knowledge, unique among open-source spectroscopy software. To appreciate this contribution, it is instructive to contrast the engine against existing computational paradigms:

- Existing spectroscopy tools (lmfit, fitspy, fityk, PyFasma, RamanSPy): Existing open-source curve-fitting tools rely on sequential, per-spectrum optimization loops. Whether implemented via Python iteration (fitspy, PyFasma) or C++ optimization (fityk), they optimize one spectrum at a time. None of them formulate the fitting of N spectra as a single batched least-squares problem solved through vectorized array operations.
- Deep learning frameworks (PyTorch, TensorFlow): While batched Levenberg-Marquardt implementations exist in machine learning repositories, they rely on automatic differentiation (autograd) designed for neural network training. They are not tailored for evaluating closed-form spectroscopic line shapes (e.g., Gaussian, Lorentzian) or for managing independent per-spectrum convergence, where each spectrum may converge at a different iteration.

Table 3 : Comparative benchmarking timing results on different datasets, tested on a PC equipped with an Intel Core i5-8365U CPU. Depending on the size of the hyperspectral dataset and the complexity of the fit model (number of peaks), SPECTROview can yield speedups of up to ~60×.

	MoS_2 flake	**Array of pixel**	**4 x WaferMap**	**CL map**
Map size (number of pixel)	1520	3721	196	16384
Number of peak(s)	3	1	6	1
Fitting time (s) : other*	22	31	33	192
Fitting time (s) : VBF engine	1.3	0.5	1.9	5.3
Performance boost	**16×**	**62×**	**17×**	**36×**

* lmfit sequential per-spectrum approach

The VBF Engine bridges this methodological gap by adapting the concept of batched optimization into a lightweight, pure-NumPy architecture. Rather than iterating over spectra in a Python loop, the engine assembles the N Jacobian matrices and normal-equation systems as three-dimensional arrays and solves them in a single vectorized LAPACK dispatch. Analytical Jacobians (derived in closed form for each supported line shape) replace the numerical finite-

difference approximations used by most tools, further reducing computational cost. A per-spectrum convergence mask progressively excludes converged spectra from subsequent iterations, ensuring that early convergers incur near-zero computational overhead. This fully vectorized strategy yields speedups of up to 60× compared to sequential per-spectrum methods (Table 3), even on a standard workstation equipped a low-power CPU (e.g., Intel Core i5-8365U).

4.2 Reproducibility and availability

SPECTROview is released under the GNU General Public License v3.0 and hosted on GitHub at https://github.com/CEA-MetroCarac/SPECTROview. The repository includes source code, example datasets. The software is registered on PyPI and can be installed or updated with a single command:

- *pip install spectroview*
- pip install –upgrade spectroview

Comprehensive documentation is available at https://CEA-MetroCarac.github.io/SPECTROview/ which includes a “User Manual” for standard users and a “Developer Guide” for those wishing to contribute to project.

SPECTROview is currently deployed within the Platform for Nanocharacterisation (PFNC) laboratories and the cleanrooms of CEA-LETI (Grenoble, France) for routine characterization and metrology of advanced semiconductor materials via Raman spectroscopy and/or photoluminescence [14-17]. It is now fully available to the broader scientific community.

5. Conclusions

SPECTROview provides an integrated, open-source solution for spectroscopic data processing, curve fitting, and visualization. By combining an interactive, highly optimized graphical user interface with a high-performance batched fitting engine, it addresses a practical gap in the open-source tooling available to researchers working with spectroscopic and hyperspectral datasets.

The underlying MVVM software architecture, workspace persistence, and extensible peak-model registry facilitate both daily laboratory workflows and long-term research reproducibility. Future development directions include support for additional spectroscopic file formats, GPU-accelerated tensor fitting, and enhanced multivariate analysis capabilities. Community contributions remain highly encouraged via the GitHub repository.

CRediT authorship contribution statement

- **Van Hoan Le**: Writing original draft, Writing review & editing, Testing, Validation, Conceptualization, Software.

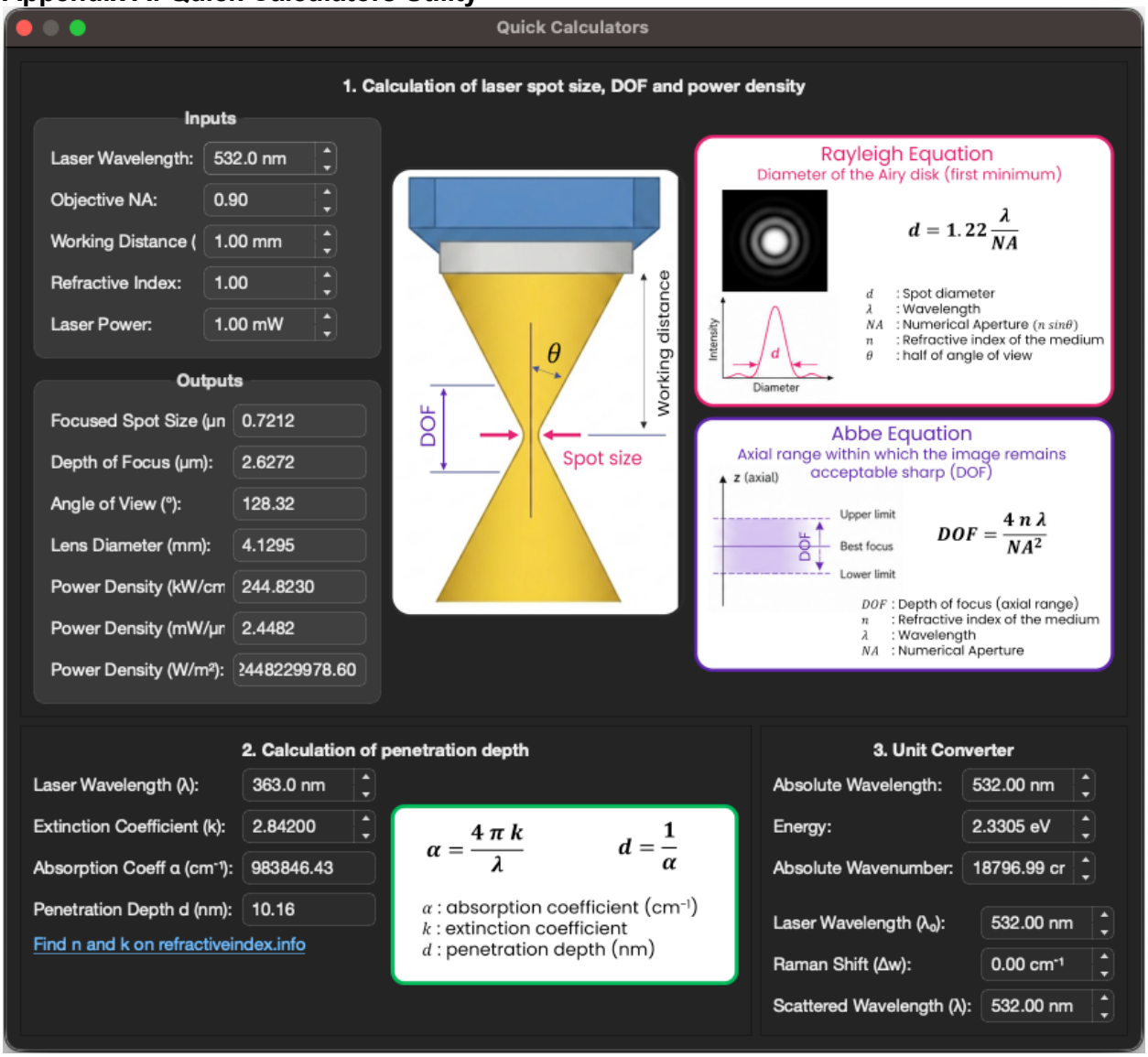
Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work, carried out on the Platform for Nanocharacterisation (PFNC), was supported by the "Recherche Technologique de Base" and "France 2030 - ANR-22-PEEL-0014" programs of the French National Research Agency (ANR). The authors gratefully acknowledge Patrick Quéméré for his assistance during the initial phase of the project.

Appendix A: Quick Calculators Utility



Listing 1: User interface of Quick Calculators Utility. More detailed on this feature at Github documentation site: here.

Appendix B: Supported Plot Styles.

References

[1] M. Wojdyr, *J. Appl. Crystallogr.* **2010**, *43*, 1126.

[2] D. Georgiev, S. V. Pedersen, R. Xie, Á. Fernández-Galiana, M. M. Stevens, M. Barahona, *Anal. Chem.* **2024**, *96*, 8492.

[3] E. Pavlou, N. Kourkoumelis, *The Analyst* **2025**, *150*, 3112.

[4] M. Newville, T. Stensitzki, D. B. Allen, A. Ingargiola, **2014**, DOI 10.5281/zenodo.11813.

[5] P. Quéméré, *J. Open Source Softw.* **2024**, *9*, 5868.

[6] The Qt Company, "Qt for Python (PySide6)," can be found under https://doc.qt.io/qtforpython-6/, **2024**.

[7] D. Erb, **2026**, pybaselines: A Python library of algorithms for the baseline correction of experimental data, https://github.com/derb12/pybaselines.

[8] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, *Nat. Methods* **2020**, *17*, 261.

[9] D. W. Marquardt, *J. Soc. Ind. Appl. Math.* **1963**, *11*, 431.

[10] E. Anderson, Z. Bai, C. Bischof, L. S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, D. Sorensen, *LAPACK Users' Guide*, Society For Industrial And Applied Mathematics, **1999**.

[11] M. L. Waskom, *J. Open Source Softw.* **2021**, *6*, 3021.

[12] J. D. Hunter, *Comput. Sci. Eng.* **2007**, *9*, 90.

[13] D. D. Lee, H. S. Seung, *Nature* **1999**, *401*, 788.

[14] Z. Szekrényes, V.-H. Le, P. Bellanger, L. Badeeb, B. Éles, A. I. Faragó, E. Nolot, D. Barge, M. Gallard, J.-M. Hartmann, P. Hauchecorne, V. Loup, J. Sturm, A. Sediri, R. Bolla, R. Petrovszki, B. Gombkötő, B. Somogyi, G. Nádudvari, A. Pongrácz, M. Dallery, N. Laurent, in *2025 36th Annu. SEMI Adv. Semicond. Manuf. Conf. ASMC*, **2025**, pp. 1–4.

[15] N.-P. Tran, F. Milesi, V.-H. Le, L.-D. Mohgouk Zouknak, P. Dezest, Ph. Rodriguez, L. Brunet, B. Duriez, M.-C. Cyrille, C. Fenouillet-Beranger, *Solid-State Electron.* **2025**, *229*, 109196.

[16] D. Barge, M. Gallard, J.-M. Hartmann, F. Fournel, V. Loup, F. Mazen, E. Nolot, P. Hauchecorne, J. Sturm, V. H. Le, I. Huyet, D. Delprat, F. Boedt, F. Servant, *Solid-State Electron.* **2026**, *232*, 109307.

[17] D. Monteil, Y. Mazel, F. Deprat, E. Prevost, R. Duru, D. L. Cunff, E. Nolot, V. H. Le, P. Quéméré, M. Mermoux, **2026**.